

Live Lecture – Week 5

- Last week: Index Construction
 - This week: Index Compression
 - Next week (Flexi week): **No** lectures. Consultations and Ed forums are still available.
-
- Today: Heaps' law, Zipf's law, Shannon entropy.
 - Today: The Programming Project spec.

Recall Reuters RCV1

■ symbol	statistic	value
■ N	documents	800,000
■ L	avg. # tokens per doc	200
■ M	terms (= word types)	~400,000
■	avg. # bytes per token (incl. spaces/punct.)	6
■	avg. # bytes per token (without spaces/punct.)	4.5
■	avg. # bytes per term	7.5
■	non-positional postings	100,000,000

Vocabulary vs. collection size

- How big is the term vocabulary?
 - That is, how many distinct words are there?
- Can we assume an upper bound?
 - Not really: At least $70^{20} = 10^{37}$ different words of length 20
- In practice, the vocabulary will keep growing with the collection size
 - Especially with Unicode 😊

Vocabulary vs. collection size

- Heaps' law: $M = kT^b$
- M is the size of the vocabulary, T is the number of tokens in the collection
- Typical values: $30 \leq k \leq 100$ and $b \approx 0.5$
- In a log-log plot of vocabulary size M vs. T , Heaps' law predicts a line with slope about $\frac{1}{2}$
 - It is the simplest possible relationship between the two in log-log space
 - An empirical finding ("empirical law")

Heaps' Law

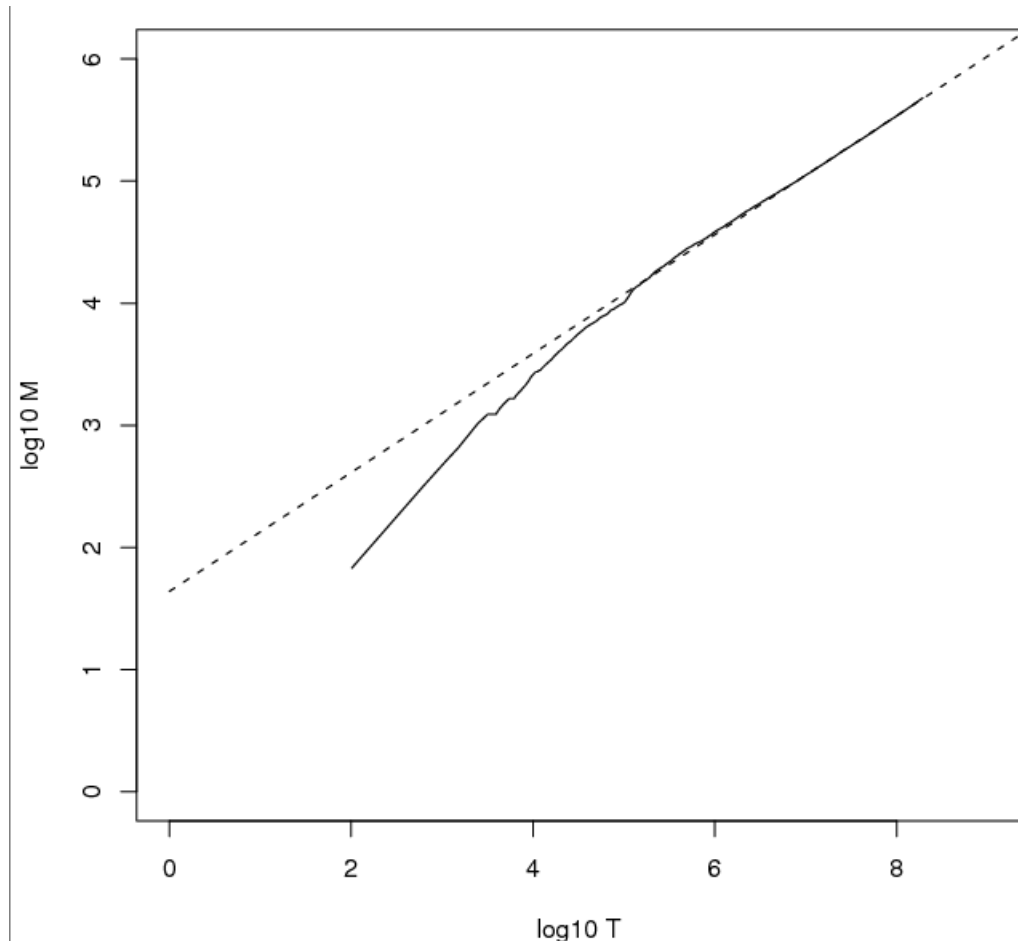
For RCV1, the dashed line
 $\log_{10} M = 0.49 \log_{10} T + 1.64$
is the best least squares fit.

Thus, $M = 10^{1.64} T^{0.49}$ so $k = 10^{1.64} \approx 44$ and $b = 0.49$.

Good empirical fit for
Reuters RCV1 !

For first 1,000,020 tokens,
law predicts 38,323 terms;
actually, 38,365 terms

Fig 5.1 p81



Exercises

- What is the effect of including spelling errors, vs. automatically correcting spelling errors on Heaps' law?
- Compute the vocabulary size M for this scenario:
 - Looking at a collection of web pages, you find that there are 3000 different terms in the first 10,000 tokens and 30,000 different terms in the first 1,000,000 tokens.
 - Assume a search engine indexes a total of 20,000,000,000 (2×10^{10}) pages, containing 200 tokens on average
 - What is the size of the vocabulary of the indexed collection as predicted by Heaps' law?

Exercises

- What is the effect of including spelling errors, vs. automatically correcting spelling errors on Heaps' law?
- Compute the vocabulary size M for this scenario:
 - Looking at a collection of web pages, you find that there are 3000 different terms in the first 10,000 tokens and 30,000 different terms in the first 1,000,000 tokens.
 - Assume a search engine indexes a total of 20,000,000,000 (2×10^{10}) pages, containing 200 tokens on average
 - What is the size of the vocabulary of the indexed collection as predicted by Heaps' law?

Zipf's law

- Heaps' law gives the vocabulary size in collections.
- We also study the relative frequencies of terms.
- In natural language, there are a few very frequent terms and very many very rare terms.
- Zipf's law: The i th most frequent term has frequency proportional to $1/i$.
- $cf_i \propto 1/i = K/i$ where K is a normalizing constant
- cf_i is collection frequency: the number of occurrences of the term t_i in the collection.

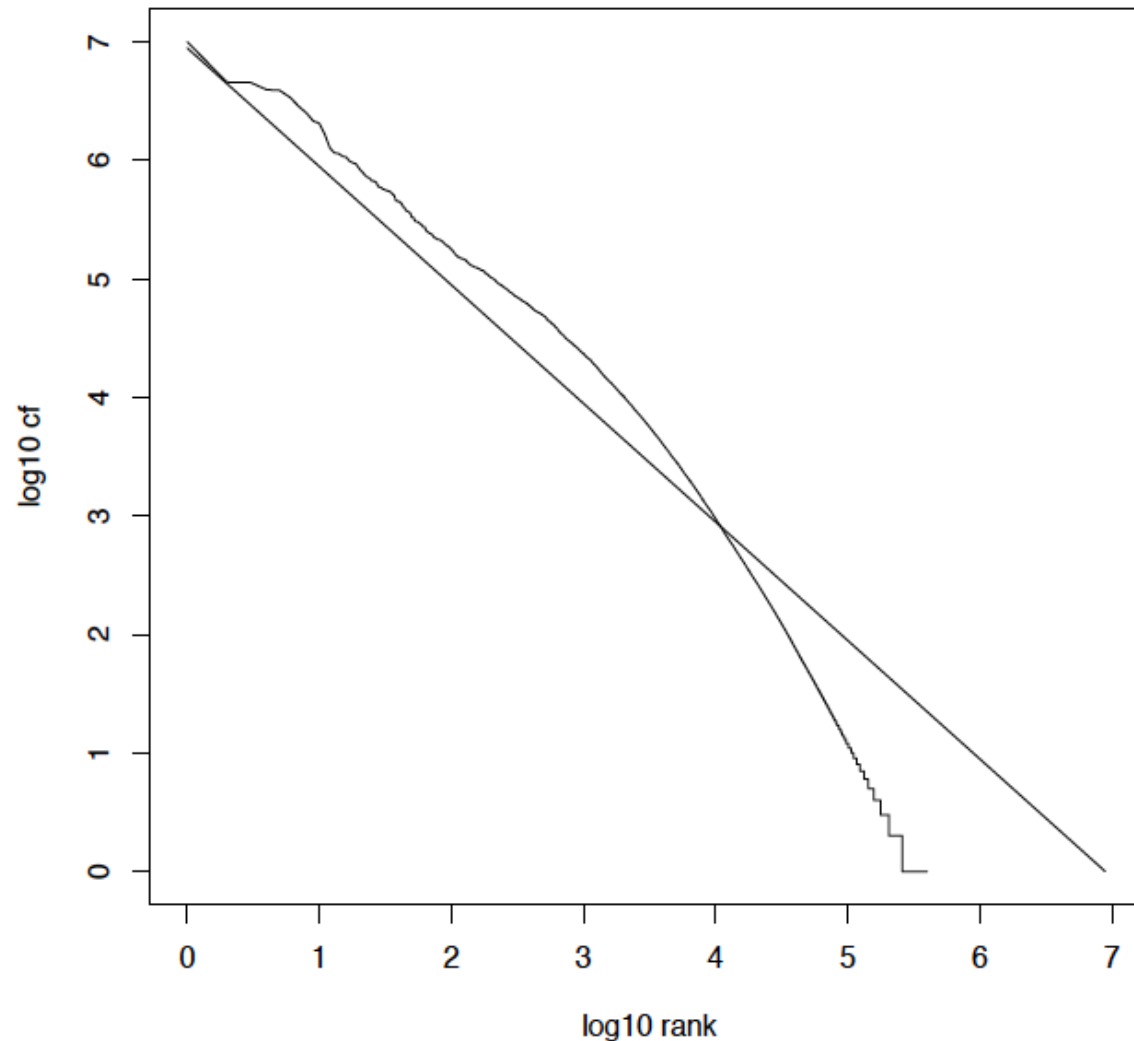
Zipf consequences

- If the most frequent term (*the*) occurs cf_1 times
 - then the second most frequent term (*of*) occurs $cf_1/2$ times
 - the third most frequent term (*and*) occurs $cf_1/3$ times ...
- Equivalent: $cf_i = K/i$ where K is a normalizing factor, so
 - $\log cf_i = \log K - \log i$
 - Linear relationship between $\log cf_i$ and $\log i$
- Another power law relationship

Zipf consequences

- If the most frequent term (*the*) occurs cf_1 times
 - then the second most frequent term (*of*) occurs $cf_1/2$ times
 - the third most frequent term (*and*) occurs $cf_1/3$ times ...
- Equivalent: $cf_i = K/i$ where K is a normalizing factor, so
 - $\log cf_i = \log K - \log i$
 - Linear relationship between $\log cf_i$ and $\log i$
- Another power law relationship

Zipf's law for Reuters RCV1



Measure of information

- Consider symbols s_i and the probability of occurrence of each symbol $p(s_i)$
- In case of fixed-length coding , smallest number of bits per symbol needed is
 - $L \geq \log_2(N)$ bits per symbol
 - Example: Message with 5 symbols need 3 bits ($L \geq \log_2 5$)

Variable length coding

- Also known as entropy coding
 - The number of bits used to code symbols in the alphabet is variable
 - E.g. Huffman coding, Arithmetic coding

Entropy

- What is the minimum number of bits per symbol?
- Answer: Shannon's result – theoretical minimum average number of bits per code word is known as Entropy (H)

$$\sum_{i=1}^n -p(s_i) \log_2 p(s_i)$$

Entropy example

- Alphabet $S = \{A, B\}$
 - $p(A) = 0.4$; $p(B) = 0.6$
- Compute Entropy (H)
 - $-0.4 \cdot \log_2 0.4 + -0.6 \cdot \log_2 0.6 = .97$ bits
- Maximum uncertainty (gives largest H)
 - occurs when all probabilities are equal

Example: ASCII

0	nul	1	soh	2	stx	3	etx	4	eot	5	enq	6	ack	7	bel
8	bs	9	ht	10	nl	11	vt	12	np	13	cr	14	so	15	si
16	dle	17	dc1	18	dc2	19	dc3	20	dc4	21	nak	22	syn	23	etb
24	can	25	em	26	sub	27	esc	28	fs	29	gs	30	rs	31	us
32	sp	33	!	34	"	35	#	36	\$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	\	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124		125	}	126	~	127	del

ASCII

- Example: SPACE is 32 or 00100000. 'z' is 122 or 01111010
- 256 symbols, assume same probability for each
- $P(s) = 1/256$
- Optimal length for each char is $\log 1/P(s)$
 $\Rightarrow \log 256 = 8$ bits

Decodable

A code is

- distinct if each codeword can be distinguished from every other (mapping is one-to-one)
- uniquely decodable if every codeword is identifiable when immersed in a sequence of codewords

Example

- A: 1
- B: 10
- C: 11
- D: 101
- To encode ABCD: 11011101
- To decode 11011101: ?

Uniquely decodable

- Uniquely decodable is a prefix free code if no codeword is a proper prefix of any other
- For example $\{1, 100000, 00\}$ is uniquely decodable, but is not a prefix code
 - consider the codeword $\{...10000000001...\}$
- In practice, we prefer prefix code (why?)

Example

S	Code
a	00
b	01
c	10
d	110
e	111

Example

S	Code
a	00
b	01
c	10
d	110
e	111

0100010011011000

Example

S	Code
a	00
b	01
c	10
d	110
e	111

0100010011011000

b a b a d d a

Q&A

Project - Ranked Retrieval

- A search engine that ranks the output documents based on the promixity of the matching terms.
- A search query = space-separated search terms
- A search term may contain any numeric digits or uppercase/lowercase letters, and will not contain any punctuations.
- Each matching doc contains terms that match all the search terms
- Need to implement an “indexer” + “search”
- Need an inverted index with positional information.

Term matching

- Search is case insensitive.
- Full stops for abbreviations are ignored. e.g., U.S., US are the same.
- Singular/Plural is ignored. e.g., cat, cats, cat's, cats' are all the same.
- Tense is ignored. e.g., breaches, breach, breached, breaching are all the same.
- A sentence can only end with a full stop, a question mark, or an exclamation mark.
- Numeric tokens such as years, integers should be indexed accordingly and searchable.
- ...more

1000 small docs from Reuters

```
$ cd ~cs6714/reuters/data
```

```
$ wc *
```

```
57  486  2966 1
```

```
26  207  1328 10
```

```
... ..
```

```
45  385  2481 983
```

```
19  148   922 988
```

```
23  165  1185 991
```

```
33186 270017 1727910 total
```

```
$
```

The indexer & The search

```
$ python3 index.py ~cs6714/reuters/data ./MyTestIndex
```

Total number of documents: 1000

Total number of tokens: 260759

Total number of terms: 9509

```
$
```

```
$ python3 search.py ~/Proj/MyTestIndex
```

Apple

1361

Malaysia

311

908

... ..

Ranking

1. by its minimum sum of the proximity distances between the matching terms (left-to-right by query terms) in an ascending order,
2. then by the number of matching terms occurring in the same order as the search terms,
3. then by the numeric values of the documentIDs (e.g., 72 will be output before 125)

Ranking (examples)

<i>DocID</i>	<i>Content</i>
1	apple durian cherry bread egg fennel garlic ham
2	bread garlic ham
3	egg bread cherry apple egg fennel ham garlic bread
4	ham garlic bread

Query: **garlic bread**

Expected output:

<i>DocID</i>	<i>Distance</i>
3	0
4	0
2	0
1	2

Query: **egg ham bread**

Expected output:

<i>DocID</i>	<i>Distance</i>
3	1+1=2
1	2+3=5

Displaying matching lines

```
$ python3 search.py ~/Proj/MyTestIndex
> bank expect distribution
> 3077
    The bank said it expects the distribution will be made in
> 4019
    It said it did not know when distributions would be made.
    The bank said it expected to report positive earnings in
> 4367
    Closing is expected to take place in early April and the
    The partnership will acquire the refining and distribution
    facility with U.S. and foreign banks to finance inventories and
> 875
    prepared to negotiate a new distribution based on objective
    bank debt, have increased political pressure on the country to
    The expected drop in prices could result in losses of as
> AUStralia Technology
> 3454
    marketing of high-technology smelting processes invented in
    Australia, notably the Siromelt Zinc Fuming Process.
> apple
> 1361
    The department said stocks of fresh apples in cold storage
$
```

Partial marks (for some queries)

Precision

Recall

$$\text{F-measure} = 2 \times (\text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$$

No partial marks for queries starting with '>'

Ans	Search1	Search2	Search3	Search4	Search5	Search6
1	1	3	2	1	1	1
2	2	2	1	2	2	2
3	3	1	4	3	3	3
4			3	4	5	5
				5	4	6

Their F-measure based on their precision and recall are calculated as below:

```

Search1 - recall: 0.75 precision: 1.00 fmeasure: 0.86
Search2 - recall: 0.25 precision: 0.33 fmeasure: 0.28
Search3 - recall: 0.50 precision: 0.50 fmeasure: 0.50
Search4 - recall: 1.00 precision: 0.80 fmeasure: 0.89
Search5 - recall: 1.00 precision: 0.80 fmeasure: 0.89
Search6 - recall: 0.75 precision: 0.60 fmeasure: 0.67

```

If this search query is worth 2 pts, Search6 will get $0.67 * 2 = 1.34$ pts.

A sanity test script

```
$ ~cs6714/reuters/sanity
```

Indexing starts...it may take up to 60 secs

10 search queries start...

Query 1-7 uses F-Measure; Query 8-10 is either correct or wrong.

query1 - Recall: 1.00 Precision: 1.00 Fmeasure: 1.00

query2 - Recall: 1.00 Precision: 1.00 Fmeasure: 1.00

... ..

query8 - Correct

query9 - Correct

query10 - Wrong

You can find the corresponding queries and expected results at:

```
cd ~cs6714/reuters/test
```

```
$
```

Python3

- On CSE Linux machines
- Use Python3
- Use Standard Library + NLTK
 - Read the spec for the notes regarding **nltk.download**

Submission

- Due: Tue 7th Nov 12:00pm (noon)
- give cs6714 proj *.py
- 6714 classrun -check proj
 - **Important**: Some students didn't check & missed some files & got 0s previously

No live lecture on Tue 7th Nov (wk 9)

- The day that the project is due.
- Scheduled content has been brought forward & made as pre-recordings, so content related to the project has been / will be covered early.
 - For example, slides & pre-recorded videos for week 7 are available now.
- A reminder will be posted on WebCMS3 Notices in wk 8.

Q&A